

Typshade Documentation

Version 0.1.0

Eito Yoneyama

2026-05-07

Abstract

Typshade is a Typst-native package for multiple-sequence alignment figures. It reimplements the practical feature set of TeXshade for shading and labeling nucleotide and peptide alignments, while exposing the interface as Typst-native building blocks: purpose-level recipes, named arguments, command helper lists, and reusable helper functions.

Contents

1	Design Goals	3
2	Package Overview	3
2.1	Package Contents	3
2.2	System Requirements	3
3	Alignment Input Files	3
3.1	MSF Files	4
3.2	ALN Files	4
3.3	FASTA Files	4
3.4	Supplemental Files	4
4	Processing Model	5
5	Quick Start	5
6	Why The UI Is Different	5
7	Feature Overview	6
7.1	Predefined Shading Modes	6
7.2	Graphs And Color Scales	6
7.3	Secondary Structures	6
7.4	Fingerprints	7
7.5	Sequence Logos And Subfamily Logos	7
7.6	Single-Sequence Figures	7
7.7	Customization Surface	8
8	Worked Examples	8
8.1	Identity Shading With A Ruler	8
8.2	Similarity Shading With Motif Labels	9
8.3	Functional Shading	9
8.4	Sequence Logo Without Sequence Rows	10
8.5	Subfamily Logo	11
8.6	Structure And Topology Tracks	11
8.7	PDB-Based Selection	13
8.8	Translation And Complement Features	13
9	Public API Layers	14
10	Figure Recipes	14
10.1	Example: Motif Map	14
11	Command Helpers	15
11.1	Example: Publication Figure	15
11.2	Example: Motifs And Graphs	16
12	Helper Function API	16

13	Detailed User Guide	17
13.1	The shade Function	17
13.2	Recipe Options	18
13.3	Selection Syntax	18
13.4	Feature Rows	18
13.5	Colors And Palettes	19
13.6	Typography And Layout	19
13.7	Pairwise Similarity And Identity	19
14	TeXshade Compatibility Reference	20
14.1	Reading The Migration Tables	20
14.2	Core Environment And Input	20
14.3	Recommended Figure-Level Replacements	20
14.4	Shading, Similarity, And Scoring	21
14.5	Sequence Lines, Labels, Numbering, Gaps	21
14.6	Consensus, Rulers, Logos, And Legend	22
14.7	Regions, Motifs, Features, Graphs, Translation	23
14.8	Sequence Visibility, Ordering, Separators, Layout	24
14.9	Structure Tracks And PDB Selections	25
14.10	Typography, Themes, And Presets	25
14.11	Utility And Analysis Functions	26
14.12	External Scripts And Side Effects	27
14.13	Complete Feature Coverage Summary	27
15	Public API Index	28
16	Public API Example Catalog	29
16.1	Rendering, Recipes, Presets, Themes, And Colors	29
16.2	Data, Inspection, And Analysis	29
16.3	Scoring, Shading, And Sequence-Type Controls	30
16.4	Windows, Labels, Numbering, Rulers, And Gaps	31
16.5	Consensus, Logos, Legends, And Swatches	32
16.6	Regions, Domains, Motifs, Marks, Graphs, And PDB Selections	33
16.7	Sequence Visibility, Ordering, Layout, Typography, And Spacing	34
16.8	Translation, Structure Tracks, And Sequence Utilities	35
17	Typst-Specific Improvements	36
18	Known Typst Differences	37
19	License	37

1 Design Goals

- Preserve TeXshade-style bioinformatics capabilities: shading, consensus, numbering, rulers, regions, features, graphs, sequence logos, subfamily logos, and structure tracks.
- Prefer Typst-native APIs over TeX macro compatibility.
- Make the common workflow short and readable through `shade(..., figure: (...))`.
- Keep low-level command helpers for exact, reproducible control.
- Avoid hidden global state. A figure should mostly describe itself where it is rendered.

2 Package Overview

Typshade is not an aligner. Like TeXshade, it assumes that a scientific alignment program has already produced an aligned nucleotide or peptide sequence file. Typshade then turns that alignment into publication-quality Typst content: colored residue cells, labels, rulers, consensus rows, logos, feature tracks, graphs, legends, and optional structure/topology annotations.

The package is organized around one rendering entry point, `shade`, plus a set of declarative helpers. The most important distinction from TeXshade is that state is represented by Typst values rather than by TeX macro side effects. This has practical consequences:

- Figure settings can be stored in variables, passed to functions, imported from other Typst files, and reviewed as data.
- The order of low-level commands still matters where the biology requires it, but purpose-level recipes reduce order sensitivity for common figures.
- Inspection helpers return Typst content/data instead of writing to the TeX log.
- Side-effect features such as exporting Chimera/Pymol command files are intentionally separated from document rendering.

2.1 Package Contents

Component	Purpose
package/lib.typ	Public package entry point. Import this file locally or as <code>@preview/typshade</code> .
internal/interface	Public-facing API modules: rendering, recipes, tracks, annotations, controls, inspection, and presets.
internal/model	Parsing, palettes, logo calculations, PDB selection helpers, and text-style helpers.
internal/render	Alignment table rendering, feature rows, graph tracks, and structure rows.
docs/documentation.typ	This manual and TeXshade-to-Typshade correspondence reference.
examples	Small runnable examples for package development and regression checks.

2.2 System Requirements

Typshade is a Typst package. It requires a Typst compiler compatible with the package metadata in `typst.toml`. It does not require LaTeX, `color.sty`, PostScript drivers, `pstricks`, or TeX memory tuning.

The package can read files that Typst is allowed to access under the active `--root`. When compiling local examples, make sure the project root includes the alignment and annotation files:

```
typst compile --root /path/to/project doc.typ out.pdf
```

3 Alignment Input Files

Typshade follows the TeXshade model: input files contain already-aligned sequences. It does not compute the alignment. Use external tools such as Clustal, MUSCLE, MAFFT, T-Coffee, or a domain-specific pipeline to create the alignment first, then read the result in your document and pass the bytes to `shade`.

The `format:` argument defaults to `auto`, but explicit formats are recommended when using `read(..., encoding: none)` because bytes do not carry a filename extension:

```
#let msf = read("alignment.msf", encoding: none)
#let aln = read("alignment.aln", encoding: none)
```

```
#let fasta = read("alignment.fasta", encoding: none)

#shade(msf, format: "msf")
#shade(aln, format: "aln", seq-type: "P")
#shade(fasta, format: "fasta", seq-type: "N")
```

3.1 MSF Files

MSF is the richest supported input format because it can carry sequence names, sequence type, lengths, and header metadata. Typshade uses the sequence names and aligned sequence blocks. If the MSF header declares peptide/nucleotide type, **seq-type: auto** can usually infer it.

Typical use:

```
#shade(
  "AQPpro.MSF",
  figure: publication(region: "80..112"),
)
```

3.2 ALN Files

ALN/Clustal-like files often lack an explicit sequence type. In such cases, pass **seq-type: "P"** for peptide alignments or **seq-type: "N"** for nucleotide alignments. Minimal ALN files are acceptable as long as sequence names and aligned blocks are recoverable.

```
#shade(
  "AQP2spec.ALN",
  seq-type: "P",
  commands: (
    diverse(),
    window(1, "77..109"),
    ruler("top", sequence: 1),
  ),
)
```

3.3 FASTA Files

FASTA files are supported for aligned sequences. Each record begins with >, and the first word is used as the default sequence name. FASTA does not encode alignment type, so explicit **seq-type:** is recommended for reproducible output.

```
#shade(
  "alignment.fasta",
  seq-type: "N",
  figure: logo-analysis(colors: "nucleotide"),
)
```

3.4 Supplemental Files

Some features need supplemental files:

File Kind	Used By	Notes
T-Coffee score data	tcoffee(source), tcoffee-scores(source)	The score data must correspond to the displayed alignment. Pass <code>read(..., encoding: none)</code> .
PHD topology/secondary files	phd-topology-track, phd-secondary-track, structure-map	Rendered as structure/feature tracks.
HMMTOP files	hmmtop-track, structures	Optional <code>hmmtop-sequence</code> selects a prediction within multi-entry files.
DSSP/STRIDE files	dssp-track, stride-track	Supports structure type filtering and appearance controls.
PDB files	pdb-point, pdb-line, pdb-plane, pdb-selection	Used for distance/plane-based residue selections.
Graph data sources	graph(..., metric: read(..., encoding: none))	Data should be normalized or have explicit ranges when needed.

4 Processing Model

TeXshade processes an environment by reading an alignment, resetting defaults, loading an optional parameter file, executing in-environment commands, then setting the alignment. Typshade keeps the same conceptual stages but expresses them as Typst values.

Step	TeXshade Model	Typshade Model
1	Analyze alignment file.	<code>shade(read(..., encoding: none), format: ...)</code> parses caller-supplied alignment bytes. Recipes may inspect the alignment before rendering.
2	Reset defaults.	The renderer starts from built-in defaults for every figure call.
3	Load parameter file.	<code>preset:</code> , <code>theme:</code> , imported Typst variables, or command arrays provide reusable settings.
4	Execute environment commands.	<code>figure:</code> , top-level arguments, <code>regions:</code> , <code>features:</code> , and <code>commands:</code> are merged explicitly.
5	Set alignment line by line.	The renderer builds Typst tables/blocks with deterministic layout.

The effective command order is:

1. `figure:` recipes or figure command fragments.
2. `preset:` and `theme:`.
3. Top-level convenience arguments such as `seq-type:`, `mode:`, `ruler:`, and `logo:`.
4. `regions:` and `features:`.
5. Explicit `commands:`.

This order lets recipes provide useful defaults while still allowing final manual overrides in `commands:`.

5 Quick Start

```
#import "@preview/typshade:0.1.0": *

#shade(
  "alignment.msf",
  theme: "screen",
  figure: motif-map(auto),
)
```

6 Why The UI Is Different

TeXshade is configured by issuing many commands inside an environment. That is natural in TeX, but it makes large figures difficult to scan because related settings are spread across an order-sensitive command stream.

TeXshade style:

```
\begin{texshade}{alignment.msf}
  \shadingmode[similar]{identical}
  \shadingcolors{blues}
  \threshold{45}
  \residuesperline{45}
  \setends{1}{80..125}
  \showruler{top}{1}
  \rulersteps{10}
  \showconsensus{bottom}
  \showsequencelogo{top}
  \shaderegion{1}{NPA}{White}{BrickRed}
  \feature{top}{1}{NXX[ST]N}{box[Yellow]}{motif}
\end{texshade}
```

Typshade style:

```
#shade(
  "alignment.msf",
```

```
figure: publication(
  similarity: "blues",
  region: "80..125",
  logo: "charge",
  motifs: (
    "NPA": (bg: "BrickRed", text: "active site"),
    "NXX[ST]N": "motif",
  ),
),
)
```

The Typshade form names the figure’s scientific intent directly. This makes documents easier to scan, review, reuse, and parameterize.

7 Feature Overview

This section mirrors the role of TeXshade’s package overview: it introduces the main capabilities before the command reference.

7.1 Predefined Shading Modes

Mode	Typshade API	Use Case
Identity	<code>identical(...)</code> or <code>scoring-mode("identical")</code>	Shade columns where a residue reaches the threshold. Useful for basic conservation figures.
Similarity	<code>similar(...)</code> or <code>scoring-mode("similar")</code>	Shade identical and chemically similar residues. This is the usual protein-alignment default.
T-Coffee	<code>tcoffee(read("scores.asc", encoding: none))</code>	Use an external T-Coffee score file to drive shading and graph tracks.
Diverse	<code>diverse(...)</code>	Highlight deviations from a reference sequence. Useful for variants and closely related homologs.
Functional	<code>functional("charge")</code> , <code>functional("hydropathy")</code> , etc.	Shade residues by biochemical property rather than only conservation.
Single sequence	<code>single-sequence(sequence: 1)</code>	Format one sequence with numbering, labels, translation/complement features, and manual regions.

Functional modes include `charge`, `hydropathy`, `structure`, `chemical`, `rasmol`, `standard area`, `accessible area`, and DNA-oriented functional shading. Custom functional group definitions use `clear-functional-groups` and `functional-group`.

7.2 Graphs And Color Scales

Feature rows can carry numerical information as bars, color scales, stacked bars, and conservation-derived plots. In TeXshade this is done through the general `\feature` command. In Typshade, the preferred high-level helper is `graph`:

```
#let alignment = read("AQPpro.MSF", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    window(1, "138..170"),
    graph("top", 1, "138..170", "conservation", kind: "bar"),
    graph("bottom", 1, "138..170", "hydrophobicity", kind: "color", options: ("GreenRed",)),
  ),
)
```

Built-in graph metrics include `conservation`, `hydrophobicity`, `molweight`, and `charge`. Graph appearance can be tuned with `bar-graph-stretch` and `color-scale-stretch`.

7.3 Secondary Structures

Typshade can overlay structure information from DSSP, STRIDE, PHD, and HMMTOP files. For ordinary figures use `structure-map` or `structures`; for exact control use the format-specific track helpers.

```
#let alignment = read("AQPpro.MSF", encoding: none)
#let topology = read("AQP1.top", encoding: none)
#let secondary = read("AQP1.phd", encoding: none)

#shade(
  alignment,
  format: "msf",
  figure: structure-map(
    1,
    topology: topology,
    secondary: secondary,
  ),
)
```

Structure visibility and appearance are controlled by `show-structure-types`, `hide-structure-types`, `structure-appearance`, `use-first-dssp-column`, and `use-second-dssp-column`.

7.4 Fingerprints

The fingerprint control gives a compact overview of long alignments by reducing residue cells to thin visual marks. It is compatible with the same shading modes, feature rows, and legends as normal alignment figures.

```
#shade(
  "AQPpro.MSF",
  commands: (
    similar(colors: "grays", all-match-threshold: 100),
    fingerprint(360),
    legend(),
  ),
)
```

7.5 Sequence Logos And Subfamily Logos

Sequence logos show residue frequency and information content per alignment position. Subfamily logos visualize residues that distinguish a subset of sequences from the rest. Typshade supports both through explicit tracks and through `logo-analysis`.

```
#shade(
  "AQPpro.MSF",
  figure: logo-analysis(
    region: "203..235",
    colors: "charge",
    subfamily: (3,),
    relevance: (threshold: 1.0, char: "*"),
  ),
)
```

Frequency correction, logo colors, logo scales, negative subfamily values, and relevance markers are available through `frequency-correction`, `logo-color`, `logo-scale`, `negative-logo-values`, and `relevance-marker`.

7.6 Single-Sequence Figures

Single-sequence display is useful for formatted ORFs, UTRs, translated regions, complements, and motif annotations. The sequence can come from a one-sequence file or from one row of a larger alignment.

```
#shade(
  "AQPDNA.MSF",
  seq-type: "N",
  commands: (
    single-sequence(sequence: 1),
    shift-single-sequence(),
    window(1, "-84..525", start: -84),
    lower(1, "-84..-1,439..525"),
    mark("bottom", 1, "390..402", fill: "Blue", text: "poly-Arg"),
  ),
)
```

```
),
)
```

7.7 Customization Surface

Most TeXshade customization categories have direct Typshade equivalents: sequence labels, numbering, rulers, gaps, regional highlighting, feature tracks, legends, captions, typography, and spacing. The important design change is that styling is local, inspectable, and composable:

```
#let paper-style = (
  typography(target: "all", size: 8pt),
  names(color: "RoyalBlue"),
  numbers(position: "right"),
  ruler("top", every: 10),
)

#let alignment = read("alignment.msfa", encoding: none)

#shade(alignment, format: "msfa", commands: paper-style)
```

8 Worked Examples

The examples below are intentionally small. They are meant to show how common TeXshade-style figures translate into idiomatic Typshade source.

8.1 Identity Shading With A Ruler

Use identity shading when exact residue conservation is the focus.

```
#let alignment = read("AQPpro.MSFA", encoding: none)

#shade(
  alignment,
  format: "msfa",
  commands: (
    identical(colors: "blues", threshold: 50, all-match-threshold: 80),
    window(1, "80..112"),
    ruler("top", sequence: 1, every: 10),
    no-consensus(),
  ),
)
```

Typeset result

```

      80      90      100     110
      |      |      |      |
AQP1.PRO T L G L L S C Q I S I L R A V M Y I I A Q C V G A I V A S A I L 112
AQP2.PRO T V A C L V G C H V S F L R A A F Y V A A Q L L G A V A G A A I L 104
AQP3.PRO T F A M C F L A R E P W I K L P I Y T L A Q T L G A F L G A G I V 112
AQP4.PRO T V A M V C T R K I S I A K S V F Y I T A Q C L G A I I G A G I L 133
AQP5.PRO T L A L L I G N Q I S L R A V F Y V A A Q L V G A I A G A G I L 105
```

The same figure can be expressed as a recipe when the intent is a standard publication panel:

```
#let alignment = read("AQPpro.MSFA", encoding: none)

#shade(
  alignment,
  format: "msfa",
  figure: publication(
    mode: "identical",
    threshold: 50,
    region: "80..112",
    conservation: false,
  ),
)
```


Typeset result

```

      80      90      100      110
      |      |      |      |
AQP1.PRO  TLGL LSCQ I I LRAVM Y I I AQCV GAI VASA IL 112
AQP2.PRO  TVAC LVGCHV S F LRAAFY VA AQL LGAVA GAAIL 104
AQP3.PRO  TFAMCFLAREPWIKLP I Y TLAQT LGAF LGAGIV 112
AQP4.PRO  TVAMVCTRK I S IAKS VFY I T AQCL GAI I GAGIL 133
AQP5.PRO  TLAL LIGNQ I S LRAVFY VA AQLV GAI A GAGIL 105
consensus ! * *      ** *****! !! *!!* ***!*

```

8.2 Similarity Shading With Motif Labels

Similarity shading is the default for many peptide alignments because it can highlight chemically similar residues even when exact identity is low.

```

#let alignment = read("AQPpro.MSF", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    similar(colors: "blues", threshold: 45),
    window(1, "80..125"),
    ruler("top", sequence: 1, every: 10),
    consensus("bottom", name: "conservation"),
    motif(1, "NPA", text: "NPA loop", fg: "White", bg: "BrickRed"),
    motif(1, "NXX[ST]N", text: "glycosylation"),
  ),
)

```

Typeset result

```

      80      90      100      110      120
      |      |      |      |      |
AQP1.PRO  TLGL LLSCQ I S I LRAVM Y I I AQCV GAI VASA IL SGIT SSL ENS LG 125
AQP2.PRO  TVAC LVGCHV S F LRAAFY VA AQL LGAVA GAAIL HEITPVE I RGD LA 117
AQP3.PRO  TFAMCFLAREPWIKLP I Y TLAQT LGAF LGAGIV FGLNE LVVSG PNG 135
AQP4.PRO  TVAMVCTRK I S IAKS VFY I T AQCL GAI I GAGIL YLVTPPSV VGLG 146
AQP5.PRO  TLAL LIGNQ I S LRAVFY VA AQLV GAI A GAGIL YW LAP NARGN LA 118
conservation ! * *      ** *****! !! *!!* ***!*      **      * **

```

For exploratory documents, `motif-map(auto)` can discover common motifs and focus the displayed region automatically.

8.3 Functional Shading

Functional shading groups residues by biochemical properties such as charge, hydrophathy, chemical class, or side-chain area.

```

#let alignment = read("AQPpro.MSF", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    functional("charge"),
    window(1, "138..170"),
    graph("top", 1, "138..170", "charge", kind: "color", options: ("ColdHot")),
    legend(),
  ),
)

```

Typeset result

■ acidic (-)
■ basic (+)

```
AQP1.PRO  GLGI[E]IIGTLQLVLCVLATT[DRRRR[D]LGGSAPL 170
AQP2.PRO  AVTV[E]LFLTMQLVLCIFAST[DE[RRG[D]NLGSPAL 162
AQP3.PRO  GFF[D]QFIGTAALIVCVLAIV[D]PNNPVP[R]GL[E]AF 186
AQP4.PRO  GLLV[E]LIITFQLVFTIFASC[DS[KRT[D]VTGSVAL 191
AQP5.PRO  AMVV[E]LILTFQLALCIFSST[DS[RRTSPVGSPAL 163
consensus *   **** !  *!*****!  ** *   !*  **
```

Custom functional modes are built from explicit group definitions:

```
#let acidic-basic = (
  functional("custom"),
  clear-functional-groups(),
  functional-group("acidic (-)", "DE", "White", "Red"),
  functional-group("basic (+)", "HKR", "White", "Blue"),
  shade-all-residues(),
)

#let alignment = read("AQPpro.MSF", encoding: none)

#shade(alignment, format: "msf", commands: acidic-basic)
```

Typeset result

■ acidic (-)
■ basic (+)

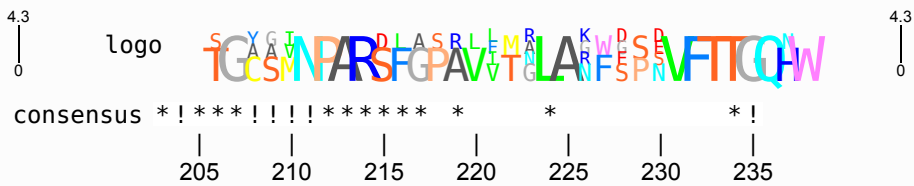
```
AQP1.PRO  GLGI[E]IIGTLQLVLCVLATT[DRRRR[D]LGGSAPL 170
AQP2.PRO  AVTV[E]LFLTMQLVLCIFAST[DE[RRG[D]NLGSPAL 162
AQP3.PRO  GFF[D]QFIGTAALIVCVLAIV[D]PNNPVP[R]GL[E]AF 186
AQP4.PRO  GLLV[E]LIITFQLVFTIFASC[DS[KRT[D]VTGSVAL 191
AQP5.PRO  AMVV[E]LILTFQLALCIFSST[DS[RRTSPVGSPAL 163
consensus *****! **!*****!*****!*****
```

8.4 Sequence Logo Without Sequence Rows

To emphasize information content rather than individual rows, hide the sequences and keep the logo, conservation row, and ruler.

```
#let alignment = read("AQPpro.MSF", encoding: none)
#shade(
  alignment,
  format: "msf",
  figure: logo-analysis(
    region: "203..235",
    colors: "charge",
    conservation: true,
    commands: (
      hide-all-sequences(),
      ruler("bottom", sequence: 3, every: 1),
    ),
  ),
)
```

Typeset result



8.5 Subfamily Logo

Subfamily logos compare a selected set of sequences against the remaining sequences. Relevance markers help identify positions with interpretable deviation.

```
#let alignment = read("AQPpro.MSF", encoding: none)

#shade(
  alignment,
  format: "msf",
  figure: logo-analysis(
    sequence: 3,
    region: "203..235",
    subfamily: (3,),
    negative: true,
    relevance: (threshold: 1.0, char: "*", color: "Black"),
    commands: (
      hide-all-sequences(),
      subfamily-logo-name("AQP3", negative-name: "others"),
    ),
  ),
)
```

Typeset result



8.6 Structure And Topology Tracks

Structure tracks can be attached either manually or through `structure-map`.

```
#let alignment = read("AQPpro.MSF", encoding: none)
#let topology = read("AQP1.top", encoding: none)
#let secondary = read("AQP1.phd", encoding: none)

#shade(
  alignment,
  format: "msf",
  figure: structure-map(
    1,
    topology: topology,
    secondary: secondary,
    region: "1..120",
  ),
)
```

Typeset result

		10		20		30		40																																		
AQP1.PRO	1	M	A	S	E	I	K	K	L	F	W	R	A	V	V	A	E	F	L	A	M	T	L	F	V	F	I	S	I	G	S	A	L	G	F	N	Y	P	L	E	40	
AQP2.PRO	1	M	W	.	E	L	R	S	I	A	F	S	R	A	V	L	A	E	F	L	A	T	L	L	F	V	F	F	G	L	G	S	A	L	Q	W	A	.	.	.	S	36
AQP3.PRO	1	M	.	.	.	.	.	.	.	L	L	R	Q	A	L	A	E	C	L	G	T	L	I	L	V	M	F	G	C	G	S	V	A	Q	V	V	L	S	R	G	44	
AQP4.PRO	1	M	S	D	G	V	W	T	Q	A	F	W	K	A	V	T	A	E	F	L	A	M	L	I	F	V	L	L	S	V	G	S	T	I	N	W	G	.	.	.	G	61
AQP5.PRO	1	M	K	.	E	V	C	S	L	A	F	F	K	A	V	F	A	E	F	L	A	T	L	I	F	V	F	F	G	L	G	S	A	L	K	W	P	.	.	.	S	37
consensus		!	*							*	*	*	*	*	!	!	*	!	*	*	*	*	!	*	*	*	*	*	!	!	*	*	*	!	!	*	*					
		50		60		70		80																																		
AQP1.PRO	41	R	N	Q	T	L	V	Q	D	N	V	K	V	S	L	A	F	G	L	S	I	A	T	L	A	Q	S	V	G	H	I	S	G	A	H	S	N	P	A	V	T	80
AQP2.PRO	37	S	.	.	.	.	P	P	S	V	L	Q	I	A	V	A	F	G	L	G	I	G	I	L	V	Q	A	L	G	H	V	S	G	A	H	I	N	P	A	V	T	72
AQP3.PRO	45	T	H	.	.	.	G	G	F	L	T	I	N	L	A	F	G	F	A	V	T	L	A	I	L	V	A	G	Q	V	S	G	A	H	L	N	P	A	V	T	80	
AQP4.PRO	62	S	E	N	P	L	P	V	D	M	V	L	I	S	L	C	F	G	L	S	I	A	T	M	V	Q	C	F	G	H	I	S	G	G	H	I	N	P	A	V	T	101
AQP5.PRO	38	A	.	.	.	.	L	P	T	I	L	Q	I	S	I	A	F	G	L	A	I	G	T	L	A	Q	A	L	G	P	V	S	G	G	H	I	N	P	A	I	T	73
consensus		*	*	*	*	*	!	!	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
		90		100		110		120																																		
AQP1.PRO	81	L	G	L	L	S	C	Q	I	S	I	L	R	A	V	M	Y	I	I	A	Q	C	V	G	A	I	V	A	S	A	I	L	S	G	I	T	S	S	L	L	120	
AQP2.PRO	73	V	A	C	L	V	G	C	H	V	S	F	L	R	A	A	F	Y	V	A	A	Q	L	L	G	A	V	A	G	A	I	L	H	E	I	T	P	V	E	I	112	
AQP3.PRO	81	F	A	M	C	F	L	A	R	E	P	W	I	K	L	P	I	Y	T	L	A	Q	T	L	G	A	F	L	G	A	G	I	V	F	G	L	N	E	L	V	130	
AQP4.PRO	102	V	A	M	V	C	T	R	K	I	S	I	A	K	S	V	F	Y	I	T	A	Q	C	L	G	A	I	I	G	A	G	I	L	Y	L	V	T	P	S	V	141	
AQP5.PRO	74	L	A	L	L	I	G	N	Q	I	S	L	L	R	A	V	F	Y	V	A	A	Q	L	V	G	A	I	A	G	A	G	I	L	Y	W	L	A	P	L	N	A	113
consensus		*	*			*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*

When exact filtering is needed, add controls:

```
#let alignment = read("AQPpro.MSF", encoding: none)
#let topology = read("AQP1.top", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    structures(1, topology: topology),
    show-structure-types(
      "PHDtopo",
      ("TM", "internal", "external"),
    ),
    structure-appearance(
      "PHDtopo",
      "TM",
      "top",
      "box[Blue]:TM",
      "",
    ),
  ),
)
```

Typeset result

```

AQP1.PRO  MASEIKKKLFWRAVVAEFLAMTLFVFISIGSALGFNYPLERNQTLVQDNVKVSLAFGLSI 60
AQP2.PRO  MW.ELRSIAFSRAVLAEFLATLLFVFFGLGSALQWA...SS...PPSVLQTAVAFGLGI 52
AQP3.PRO  M.....LLRQALAECLGTLILVMFGCGSVAQVVLSRGTH...GGFLITNLAFGLFAV 60
AQP4.PRO  MSDGVWTQAFWKAVTAEFLAMLIFVLLSVGSTINWG...GSENPLPVDMLISLCFGLSI 81
AQP5.PRO  MK.EVCSLAFKAVFAEFLATLIFVFFGLGSALKWP...SA...LPTILQTSIAFGLAI 53
consensus !  *      **  ***  !!!*****!***  !!!*  *      *  ****!!*  *

AQP1.PRO  ATLLAQSVGHISGAHSNPAVT 80
AQP2.PRO  GILVQALGHVSGAHINPAVT 72
AQP3.PRO  TLAILVAGQVSGAHLNPAVT 80
AQP4.PRO  ATMVCFGHISGGHINPAVT 101
AQP5.PRO  GTLLAQALGPVSGGHINPAIT 73
consensus **  *  !***!!*!!*!!*!!

```

8.7 PDB-Based Selection

PDB selections are especially useful for showing residues near a structural site without hand-transcribing residue numbers.

```

#let alignment = read("AQPpro.MSF", encoding: none)
#let pdb = read("1J4N.pdb", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    window(1, pdb-point(pdb, 81, distance: 8, atom: "CA")),
    ruler("top", sequence: 1, every: 1),
    highlight(1, pdb-line(pdb, 81, 168), bg: "Yellow"),
  ),
)

```

Typeset result

```

      123
      |||
Alpha AEF. 3
Beta  ADF. 3
Gamma A. G. 2
seq4  AC.. 2

```

8.8 Translation And Complement Features

Single-sequence nucleotide figures can combine lowercased UTRs, complements, translations, and manual feature labels.

```

#let alignment = read("AQPDNA.MSF", encoding: none)

#shade(
  alignment,
  format: "msf",
  seq-type: "N",
  commands: (
    single-sequence(sequence: 1),
    shift-single-sequence(),
    window(1, "-84..525", start: -84),
    lower(1, "-84..-1,439..525"),
    backtranslation-label("horizontal"),
    mark("bottom", 1, "390..402", fill: "Blue", text: "poly-Arg"),
  ),
)

```

Typeset result

AQP1nuc.SEQ CCTGGGCATTGAGATCATTGGCACCCctgca 443

9 Public API Layers

Typshade has three layers. You can mix them, but most documents should start at the top.

Layer	Use When	Examples
figure: recipes	You know the purpose of the figure.	publication, motif-map, structure-map, logo-analysis
commands: helpers	You want reusable pieces or small adjustments.	similar, lines, ruler, logo, highlight
Fine-grained commands	You need precise control over a TeXshade-equivalent feature.	weight-table, residue-style, feature-rule, structure-appearance

10 Figure Recipes

figure: is the recommended API for hand-written scientific figures. Recipes expand into lower-level track, annotation, style, and scoring commands.

Recipe	Purpose	Typical Use
publication	Balanced paper-ready alignment figure.	publication(motifs: auto, logo: auto)
motif-map	Detect or highlight motifs and add a conservation graph.	motif-map(auto)
structure-map	Combine alignment, ruler, consensus, and structure tracks.	structure-map(1, topology: read("AQP1.top", encoding: none))
logo-analysis	Show sequence/subfamily logo analysis.	logo-analysis(colors: "charge", subfamily: (1, 2, 3))
overview	Compact overview for many sequences or quick reports.	overview(colors: "grays")

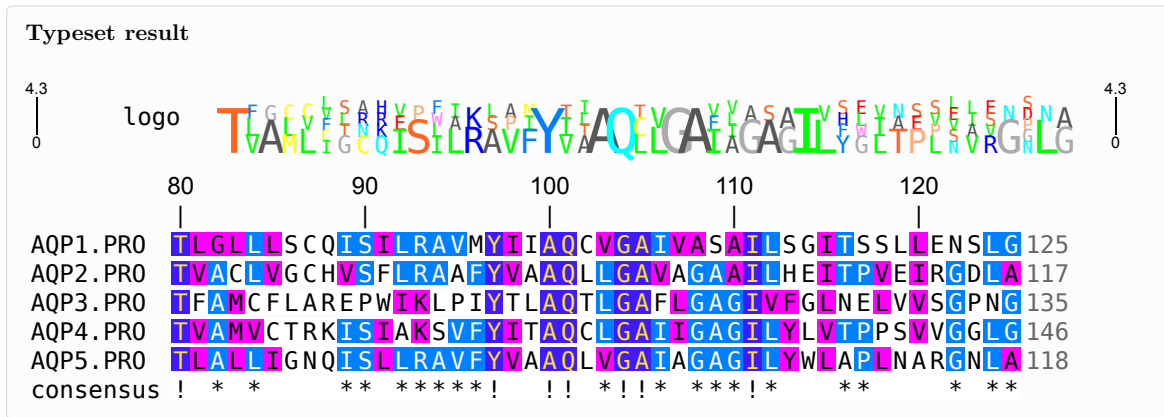
Smart defaults use the alignment data:

- **motif-map(auto)** detects common protein or nucleotide motifs.
- **region: auto** focuses around detected motifs.
- **line-length: auto** adapts to sequence count and selected region.
- **logo: auto** enables logos only when they remain readable.
- **threshold: auto** chooses a sequence-type-aware default.

10.1 Example: Motif Map

```
#let alignment = read("alignment.msfa", encoding: none)

#shade(
  alignment,
  format: "msfa",
  figure: motif-map(
    (
      "NPA": (bg: "BrickRed", text: "active site"),
      "NXX[ST]N": "glycosylation",
    ),
    region: "80..125",
    logo: "charge",
  ),
)
```



11 Command Helpers

`commands`: is useful when a recipe is too high-level and you want to assemble the visible parts yourself. It accepts helper output and low-level commands in left-to-right order.

Helper	Purpose	Example
similar, identical, diverse, functional	Choose scoring and optional colors/thresholds.	<code>similar(colors: "blues", threshold: 45)</code>
lines	Set residues per line.	<code>lines(50)</code>
window	Show a selected sequence range.	<code>window(1, "80..125")</code>
names, numbers	Configure sequence labels and numbering.	<code>names(color: "RoyalBlue")</code>
ruler, consensus, logo, legend	Add common tracks.	<code>ruler("top", every: 10)</code>
highlight, tint, emphasize	Style residue regions.	<code>highlight(1, "NPA", bg: "BrickRed")</code>
motif, mark, graph	Add feature rows and graph tracks.	<code>motif(1, "NXX[ST]N", text: "motif")</code>
structures	Add topology/secondary-structure tracks.	<code>structures(1, topology: read("AQP1.top", encoding: none))</code>
typography	Adjust text for named parts.	<code>typography(target: "names", family: "mono")</code>

11.1 Example: Publication Figure

```
#let alignment = read("alignment.msf", encoding: none)

#shade(
  alignment,
  format: "msf",
  preset: "publication",
  theme: "screen",
  commands: (
    similar(threshold: 45),
    lines(50),
    ruler("top", sequence: 1, every: 10),
    consensus("bottom", name: "conservation"),
  ),
)
```

Typeset result

```

      80          90          100          110          120
      |          |          |          |          |
AQP1.PRO TLGLLLSCQISILRAVMYIIAQC VGAIVASAIIISGIISSLL ENSLG 125
AQP2.PRO TVACLVGCHVSFLRAAFYVA AQL LGAVAGAAIILHEITPVEIRGD LA 117
AQP3.PRO TFAMCFLAREPWIKLPIYTLAQT LGAF LGAGIVFGLNELVVS GPNG 135
AQP4.PRO TVAMVCTRKISIAKSVFYITAQC LGAIIGAGILYLVTTPSVV GGLG 146
AQP5.PRO TLALLIGNQISLLRAVFYVA AQL VGAIAGAGILYWLAPLNARGN LA 118
conservation ! * *      ** *****! !! *!!* ***!* **      * **

```

11.2 Example: Motifs And Graphs

```
#let alignment = read("alignment.msf", encoding: none)
```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    window(1, "80..125"),
    highlight(1, "NPA", bg: "BrickRed"),
    motif(1, "NXX[ST]N", text: "glycosylation"),
    graph("bottom", 1, "all", "conservation", kind: "color", options: ("ColdHot",)),
  ),
)

```

Typeset result

```

AQP1.PRO TLGL L LSCQISILRAVMYIIAQC VGAIVASAIIISGIISSLL ENSLG 125
AQP2.PRO TVACLVGCHVSFLRAAFYVA AQL LGAVAGAAIILHEITPVEIRGD LA 117
AQP3.PRO TFAMCFLAREPWIKLPIYTLAQT LGAF LGAGIVFGLNELVVS GPNG 135
AQP4.PRO TVAMVCTRKISIAKSVFYITAQC LGAIIGAGILYLVTTPSVV GGLG 146
AQP5.PRO TLALLIGNQISLLRAVFYVA AQL VGAIAGAGILYWLAPLNARGN LA 118
consensus ! * *      ** *****! !! *!!* ***!* **      * **

```

12 Helper Function API

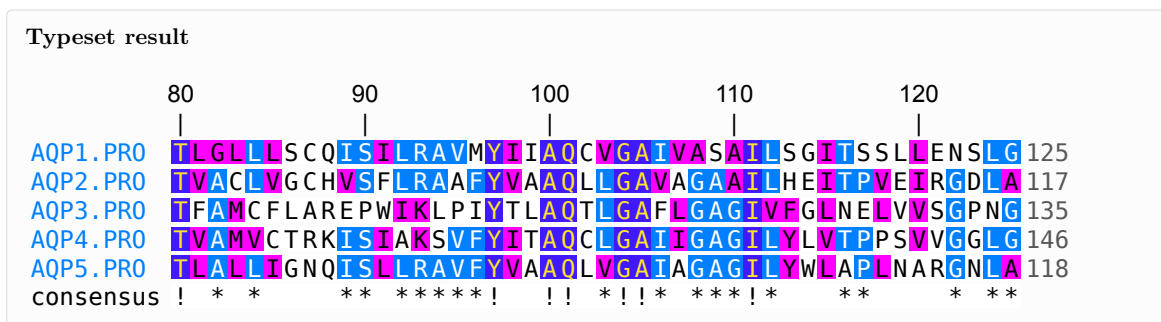
Helper functions are useful when a figure is assembled from reusable fragments.

```

#let active-site = (
  highlight(1, "NPA", fg: "White", bg: "BrickRed"),
  motif(1, "NXX[ST]N", text: "motif"),
)

#shade(
  read("alignment.msf", encoding: none),
  format: "msf",
  preset: "publication",
  theme: "screen",
  regions: active-site,
  commands: (
    scoring-mode("similar"),
    ruler-track(position: "top", sequence: 1, steps: 10),
  ),
)

```

13 Detailed User Guide

This section describes the public surface in the style of a user manual. The tables below are intentionally redundant with the compatibility reference: the guide explains how to write Typshade; the compatibility reference explains where each TeXshade command went.

13.1 The `shade` Function

`shade` is the only rendering function most documents need.

```
#shade(
  source,
  format: auto,
  figure: (),
  preset: none,
  theme: none,
  mode: none,
  option: none,
  seq-type: auto,
  residues-per-line: none,
  names: none,
  numbering: none,
  consensus: none,
  ruler: none,
  logo: none,
  subfamily-logo: none,
  legend: none,
  regions: (),
  features: (),
  commands: (),
  font: none,
  font-size: none,
)
```

Argument	Meaning
<code>source</code>	Alignment source, normally <code>read("alignment.msf", encoding: none)</code> . This is the only required argument.
<code>format</code>	<code>auto</code> by default; use explicit formats for ambiguous MSF/ALN/FASTA-like files.
<code>figure</code>	Purpose-level recipe or array of recipes/commands. This is the recommended high-level API.
<code>preset</code>	Named reusable defaults such as publication/overview-oriented settings.
<code>theme</code>	Named or custom visual theme. Themes should express visual identity, not biology.
<code>mode, option</code>	Direct scoring-mode override. Prefer <code>similar</code> , <code>functional</code> , <code>tcoffee</code> , etc. inside <code>commands</code> : when assembling manually.
<code>seq-type</code>	<code>auto</code> , <code>"P"</code> , or <code>"N"</code> depending on whether type should be inferred or forced.
<code>residues-per-line</code>	Direct line length override. Shortcut equivalent: <code>lines(count)</code> .
<code>names, numbering, consensus, ruler, logo, legend</code>	Convenience toggles or simple track declarations. Use track helpers for more control.
<code>regions, features</code>	Dedicated slots for highlight/annotation command fragments.
<code>commands</code>	Final explicit command list. Use this for low-level controls and final overrides.
<code>font, font-size</code>	Renderer-level fallback text settings. Prefer <code>typography</code> for targeted styling.

By default, rendered alignment blocks are left-aligned. Use `alignment-position("center")` or `alignment-position("right")` only when a particular figure should be optically centered or flush-right inside its containing block.

13.2 Recipe Options

Recipes are the main UI/UX improvement over TeXshade. They describe the kind of figure rather than the macro operations needed to build it.

publication option	Meaning
mode	Scoring mode, default "similar".
similarity	Color scheme, default "blues".
threshold	auto chooses a sequence-type-aware threshold; a number fixes it.
sequence	Reference sequence for region, ruler, and motif interpretation.
region	auto, none, range string, motif string, or dictionary with sequence/selection.
line-length	auto picks a readable line length; a number fixes residues per line.
ruler, every	Enable ruler and choose tick interval.
conservation	Controls consensus/conservation row.
logo	Logo color set, auto, false, or none.
motifs, highlights	Motif and region annotations.
theme, annotations, commands	Additional visual theme and final command fragments.

Recipe	Distinctive Options
motif-map	motifs, region, graph, logo, and conservation can all be auto so the alignment drives the result.
structure-map	topology, secondary, hmmtop, and hmmtop-sequence attach structure tracks to a chosen sequence.
logo-analysis	colors, subfamily, negative, and relevance configure sequence/subfamily logo analysis.
overview	names, numbers, conservation, and ruler default to compact values for large alignments.

13.3 Selection Syntax

Selections are used by windows, highlights, motifs, feature rows, graphs, and PDB helpers.

Selection Kind	Example	Meaning
Single range	"80..112"	Residues 80 through 112 in the reference sequence.
Multiple ranges	"80..90,100..110"	Union of several displayed spans.
Keyword	"all"	All available positions in the selected sequence.
Motif	"NPA"	All matching motif occurrences.
Pattern motif	"NXX[ST]N"	X matches any residue; bracket groups match any listed residue.
PDB point	pdb-point(read("1J4N.pdb", encoding: none), 81, distance: 8, atom: "CA")	Residues near a point around a PDB anchor.
PDB line	pdb-line(read("1J4N.pdb", encoding: none), 81, 168)	Residues close to a line between two anchors.
PDB plane	pdb-plane(read("1J4N.pdb", encoding: none), 66, 73, 199)	Residues close to a plane defined by three anchors.

Use `selection-preview` or `selection-table` when a motif or PDB selection is too subtle to trust by eye.

13.4 Feature Rows

The general TeXshade `\feature` command is powerful but dense. Typshade splits common cases into clearer helpers:

Task	Helper	Example
------	--------	---------

Text or boxed mark	mark	mark("top", 1, "93..93", fill: "Yellow", text: "site")
Motif highlight plus label	motif	motif(1, "NXX[ST]N", text: "glycosylation")
Region color	highlight	highlight(1, "NPA", fg: "White", bg: "BrickRed")
Tint existing shading	tint	tint(1, "158..163", intensity: "strong")
Emphasize text	emphasize	emphasize(1, "QLVLC", style: "italic")
Graph track	graph	graph("bottom", 1, "all", "conservation", kind: "color")

When you need exact TeXshade-style label strings, use the lower-level feature controls through **commands**:. The renderer still supports feature positions such as **ttttop**, **tttop**, **ttop**, **top**, **bottom**, **bbottom**, **bbbtop**, and **bbbbottom**.

13.5 Colors And Palettes

Typshade accepts the package color names used by its palettes and resolves them through Typst colors. Built-in color schemes include TeXshade-style schemes such as **blues**, **reds**, **greens**, **grays**, and **black**, plus graph scales such as **BlueRed**, **RedBlue**, **GreenRed**, **RedGreen**, **ColdHot**, **HotCold**, **WhiteBlack**, and **BlackWhite**.

Use these levels of control:

1. **theme**: for the document's visual direction.
2. **color-scheme** or scoring shortcuts for alignment categories.
3. **residue-style**, **functional-style**, **logo-color**, or **consensus-colors** for exact category/residue styling.
4. **visual-theme** when creating a reusable house style.

13.6 Typography And Layout

TeXshade exposes many short font commands because TeX font selection is macro oriented. Typshade uses targeted text controls:

Control	Purpose
typography(target: ..., family: ..., weight: ..., posture: ..., size: ...)	Convenient combined text styling.
text-family, text-weight, text-posture, text-size	Set one text attribute for one target.
text-style	Set family, weight, posture, and size together.
character-stretch, line-stretch	Tune residue cell dimensions.
block-gap, line-gap, feature-slot-space	Tune vertical spacing.
caption, short-caption	Attach figure-style captions to alignment output.

13.7 Pairwise Similarity And Identity

TeXshade exposes **\percentsimilarity**, **\percentidentity**, and **\similaritytable** outside the alignment environment. Typshade provides the same analysis as explicit data/document helpers:

```
#let alignment = read("AQPpro.MSF", encoding: none)

#percent-similarity(alignment, 1, 2, format: "msf", selection: "80..112")
#percent-identity(alignment, "AQP1.PRO", "AQP2.PRO", format: "msf")
#similarity-table(alignment, format: "msf", selection: "80..112")
```

similarity-table follows TeXshade's combined table convention: values above the diagonal are percent similarity and values below the diagonal are percent identity. Calculations use shared non-gap positions in the selected alignment span.

14 TeXshade Compatibility Reference

This section is intentionally detailed. It is the migration contract: every major TeXshade feature is listed with the corresponding Typshade API and the reason when the mapping is not literal.

Typshade does not keep TeX macro names as public names. Instead it preserves the scientific capability and exposes it through Typst-style names, typed values, recipes, and explicit command lists.

Status	Meaning
native	The feature is available through a Typst-native API and should be preferred for new documents.
control	The feature is available as a lower-level command helper, usually inside <code>commands:</code> .
recipe	The feature is normally expressed through a high-level <code>figure:</code> recipe.
data	The TeXshade command produced messages or files; Typshade exposes data/content instead.
approx	The visible behavior is implemented, but TeX box/glue details are intentionally not reproduced exactly.
omitted	The behavior is a TeX side effect, external file writer, or driver-specific hook that is not appropriate inside a Typst package.

14.1 Reading The Migration Tables

Each table row gives the TeXshade command or feature, the Typshade replacement, the support status, and migration notes. When a row lists several TeXshade commands, those commands are one feature family in TeXshade and are configured together in Typshade.

For new documents, prefer this order:

1. Use `figure:` recipes when the figure has a recognizable purpose.
2. Use shortcut helpers such as `similar`, `ruler`, `logo`, `motif`, and `structures` when you are assembling a figure manually.
3. Use lower-level controls such as `weight-table`, `residue-style`, `structure-appearance`, or `feature-slot-space` only when you need exact control over a TeXshade-equivalent detail.

14.2 Core Environment And Input

TeXshade	Typshade	Status	Migration Notes
<code>\begin{texshade}</code> <code>{file}...</code> <code>\end{texshade}</code>	<code>shade(read("file", encoding: none), format: "msf", figure: publication(...))</code>	native	The entry point is a function call. The alignment source is explicit Typst data supplied by the caller, so package rendering does not depend on package-internal file access.
TeXshade parameter files	<code>#let common = (...)</code> plus <code>commands: common</code> or <code>figure: ...</code>	native	Use Typst variables, dictionaries, arrays, imports, and presets instead of order-sensitive parameter files.
<code>seqtype</code>	<code>seq-type:</code> or <code>sequence-</code> <code>type("P") / sequence-type("N")</code>	native	Automatic detection remains available; explicit overrides are stable and local to the figure.
MSF, FASTA-like, ALN-like inputs	<code>format: auto</code> , <code>format: "msf"</code> , or explicit parser format	native	Use <code>format: auto</code> unless an ambiguous file needs an override.
TeX command stream inside the environment	<code>commands: (...)</code> , <code>regions:</code> <code>(...)</code> , <code>features: (...)</code> , <code>figure: ...</code>	native	The order is visible as Typst array order; reusable command fragments can be normal variables.
TeXshade inspection by compiling/log output	<code>alignment-summary</code> , <code>sequence-</code> <code>list</code> , <code>selection-preview</code> , <code>selection-table</code>	data	Inspection is document content/data and can be embedded in reports.

14.3 Recommended Figure-Level Replacements

TeXshade Use Case	Typshade	Status	Migration Notes
Publication alignment with shading, ruler, consensus	<code>figure: publication(...)</code>	recipe	This replaces a long TeXshade setup with a purpose-level declaration. Use <code>similarity</code> , <code>threshold</code> , <code>region</code> , <code>motifs</code> , <code>logo</code> , and <code>ruler</code> options.

Motif-centric figure	<code>figure: motif-map(auto) or motif-map(...)</code>	recipe	Automatically detects common motifs, can focus the displayed region, and can add motif labels and conservation graphs.
Structure/topology figure	<code>figure: structure-map(sequence, topology: ..., secondary: ...)</code>	recipe	Combines alignment, ruler, consensus, and topology/secondary-structure tracks.
Logo/subfamily analysis	<code>figure: logo-analysis(subfamily: (...), colors: ...)</code>	recipe	Builds sequence-logo and subfamily-logo views without manually managing all logo commands.
Dense overview of many sequences	<code>figure: overview(...)</code>	recipe	Compact defaults for high sequence counts or report appendices.

14.4 Shading, Similarity, And Scoring

TeXshade	Typshade	Status	Migration Notes
<code>shadingmode[similar]{identical}</code>	<code>identical(...), similar(...), diverse(...), functional(...), single-sequence(...), or scoring-mode(...)</code>	native	The common modes are shortcuts; <code>scoring-mode</code> is the exact lower-level control. T-Coffee uses <code>tcoffee(...)</code> or <code>tcoffee-scores(...)</code> .
<code>shadingcolors</code>	<code>color-scheme("blues") or similar(colors: "blues")</code>	native	Color scheme selection is separated from scoring mode so themes can be reused.
<code>defshadingcolors</code>	<code>shade-theme(...), visual-theme(...), color-scheme(...), and residue-style(...)</code>	native	Rather than snapshotting mutable TeX state, Typshade encourages named dictionaries and explicit style commands.
<code>nomatchresidues, similarresidues, conservedresidues, allmatchresidues</code>	<code>residue-style("nomatch", ...), residue-style("similar", ...), residue-style("conserved", ...), residue-style("allmatch", ...)</code>	control	Foreground, background, case, and emphasis style can be configured per residue category.
<code>threshold</code>	<code>threshold(45) or similar(threshold: 45)</code>	native	Controls conservation and shading threshold. Recipes accept <code>threshold: auto</code> for sequence-type-aware defaults.
<code>allmatchspecial, allmatchspecialoff</code>	<code>all-match-threshold(value: 100), disable-all-match-threshold()</code>	control	Controls whether very high-conservation columns receive special handling.
<code>hideallmatchpositions, showallmatchpositions</code>	<code>hide-all-match-positions(), show-all-match-positions()</code>	control	Supports removal/restoration of columns above the all-match threshold.
<code>shadeallresidues</code>	<code>shade-all-residues()</code>	control	Equivalent to lowering the threshold so every residue is eligible for shading.
<code>weighttable</code>	<code>weight-table("BLOSUM62"), weight-table("PAM250"), weight-table("PAM100"), weight-table("structural")</code>	control	Weighted consensus scoring is available with common TeXshade matrices.
<code>setweight</code>	<code>set-weight("A", "G", value)</code>	control	Overrides individual pair weights for custom scoring.
<code>gappenalty</code>	<code>gap-penalty(value)</code>	control	Controls how gaps contribute to weighted consensus scoring.
<code>pepgroups, pepsims</code>	<code>peptide-groups(...), peptide-similarities(...)</code>	control	Custom peptide groupings and per-residue similarity sets.
<code>DNAGroups, DNAsims</code>	<code>dna-groups(...), dna-similarities(...)</code>	control	Custom nucleotide groupings and per-base similarity sets.
<code>clearfuncgroups, funcgroup</code>	<code>clear-functional-groups(), functional-group(...)</code>	control	Defines a custom functional shading scheme.
<code>funcshadingstyle</code>	<code>functional-style(residue, fg, bg, case: ..., style: ...)</code>	control	Overrides style for a functional residue category.
T-Coffee scoring file option, <code>includeTCoffee</code>	<code>tcoffee(read("scores.tcs", encoding: none)) or tcoffee-scores(read("scores.tcs", encoding: none))</code>	native	Loads T-Coffee confidence data as a scoring mode option.

14.5 Sequence Lines, Labels, Numbering, Gaps

TeXshade	Typshade	Status	Migration Notes
<code>residuesperline, residuesperline*</code>	<code>lines(60) or residues-per-line(60)</code>	native	Controls columns per alignment block. TeXshade's starred no-width-check form is

			represented by explicit Typst layout control rather than a second function name.
setends	window(sequence, "80..125", start: ...) or sequence-window(...)	native	Selects displayed ranges by sequence and residue selection.
setdomain	domain(sequence, selection)	control	Domain selection maps to the same explicit window model. Domain-specific gap styling uses normal gap controls.
shownames, hidenames	names(position: "left"), no-names()	native	Sequence-name labels are an explicit track.
nameseq	sequence-name(sequence, "label")	control	Renames an individual sequence label.
namescolor, namecolor	names-color(color), sequence-name-color(sequences, color)	control	Global and per-sequence label colors.
hidename	hide-sequence-name(sequences)	control	Hides selected sequence labels while keeping rows.
shownumbering, hidenummering	numbers(position: "right"), no-numbers()	native	Numbering is an explicit track.
numberingcolor, numbercolor	numbering-color(color), sequence-number-color(sequences, color)	control	Global and per-sequence numbering colors.
hidenummer	hide-sequence-number(sequences)	control	Hides numbering for selected rows.
startnumber	start-number(sequence, start, selection: ...)	control	Overrides displayed numbering start and optional selection scope.
allowzero, disallowzero	allow-zero-numbering(), disallow-zero-numbering()	control	Controls whether zero can appear in numbering.
seqlength	sequence-length(sequence, length)	control	Overrides reported sequence length.
gapchar	gap-char(symbol)	control	Changes visible gap symbol.
gaprule	gap-rule(thickness)	control	Draws gaps as rules.
gapcolors	gap-colors(foreground, background)	control	Sets rule/character gap colors.
domaingaprule, domaingapcolors	domain-gap-rule(thickness), domain-gap-colors(foreground, background)	control	Available as domain-oriented aliases over the same gap styling model.
showleadinggaps, hideleadinggaps	show-leading-gaps(), hide-leading-gaps()	control	Controls whether leading gaps are visible.
stopchar	stop-char(symbol)	control	Custom stop codon/terminal symbol.
shiftsingleseq, keepsingleseqgaps	shift-single-sequence(value: ...), keep-single-sequence-gaps()	approx	Single-sequence offset behavior is represented, but TeX's unusual gap-rewriting edge cases are not treated as byte-for-byte compatibility targets.
hideresidues, showresidues	hide-residues(), show-residues()	control	Hides residue letters while preserving colored cells.
fingerprint	fingerprint(value)	approx	Provides condensed fingerprint-like rendering controls; exact TeX ultra-thin bar metrics are Typst layout approximations.

14.6 Consensus, Rulers, Logos, And Legend

TeXshade	Typshade	Status	Migration Notes
showconsensus, hideconsensus	consensus(position: "bottom"), consensus-track(...), no-consensus()	native	Adds or removes the consensus row.
nameconsensus	consensus-name("consensus")	control	Controls consensus row label.
defconsensus	consensus-symbols(none, conserved, allmatch)	control	Controls symbols used in the consensus row.
consensuscolors	consensus-colors(...)	control	Configures foreground/background colors for consensus states.
constosingleseq, constoallseqs	consensus-from-sequence(sequence), consensus-from-all-sequences()	control	Controls the basis for consensus calculation.
germanlanguage, spanishlanguage, englishlanguage	consensus-language("german"), etc.	control	Applies to document-facing consensus labels. TeX diagnostic message localization is not reproduced.

exportconsensus	alignment-data(...), parse-alignment(...), or external tooling	omitted	Typst packages should not write arbitrary external files while rendering. Export-like workflows should use explicit build scripts.
showruler, hideruler	ruler("top", sequence: 1), ruler-track(...), no-ruler(...)	native	Adds top/bottom ruler tracks.
rulersteps	ruler-steps(value, position: ...) or ruler(..., every: value)	native	Controls tick interval.
rulercolor	ruler-color(color, position: ...)	control	Controls ruler color.
namerulerpos, nameruler	ruler-marker(number, text, position: ...), ruler-name(name, position: ...)	control	Adds named ruler markers or a ruler label.
rulernamecolor, rulerspace	ruler-name-color(color), ruler-space(value)	control	Controls ruler label styling and spacing.
rotateruler, unrotateruler	rotate-ruler(position: ...), unrotate-ruler(position: ...)	approx	Typst rotates content, but TeX baseline/glue side effects are not copied exactly.
showsequencelogo, hidesequencelogo	logo(position: "top"), sequence-logo(...), no-sequence-logo()	native	Adds or removes sequence logos.
namesequencelogo	sequence-logo-name(name) or sequence-logo(name: ...)	control	Controls sequence-logo label.
showlogoscale, hidelogoscale	logo-scale(position: ..., color: ...), no-logo-scale()	control	Controls logo scale visibility and placement.
logostretch	logo-stretch(value)	control	Vertical stretch for logo display.
logocolor, clearlogocolors	logo-color(residues, color), clear-logo-colors(default: ...)	control	Per-residue logo color control.
dofrequencycorrection, undofrequencycorrection	frequency-correction(), no-frequency-correction()	control	Controls small-sample correction for logo information content.
setsubfamily, showsubfamilylogo, hidesubfamilylogo	subfamily(sequences), subfamily-logo(...), no-subfamily-logo()	native	Subfamily logo support is available either as explicit commands or through logo-analysis .
namesubfamilylogo	subfamily-logo-name(name, negative-name: ...)	control	Controls labels for positive/negative subfamily logos.
shownegatives, hidenegatives	negative-logo-values(), no-negative-logo-values()	control	Controls negative logo values.
relevance, subfamilythreshold, showrelevance, hiderelevance	relevance-threshold(value), relevance-marker(char: ..., color: ...), no-relevance-marker()	control	Controls subfamily-logo relevance marks. subfamilythreshold is treated as an older spelling for the same threshold concept.
showlegend, hidelegend	legend(color: ...), legend-track(...), no-legend()	native	Legend entries derive from functional groups and theme styling.
legendcolor, movelegend, shadebox	legend-color(color), legend-offset(dx, dy), color-swatch(color)	control	Legend placement and swatches are Typst content rather than TeX boxes.

14.7 Regions, Motifs, Features, Graphs, Translation

TeXshade	Typshade	Status	Migration Notes
shaderegion, shadeblock	highlight(sequence, selection, ...) or highlight-block(...)	native	Selections support ranges, motif-like patterns, and PDB-derived selections.
changeshadingcolors	region-color-scheme(sequence, selection, scheme)	control	Changes the color scheme for a selected region.
tintregion, tintblock	tint(sequence, selection, intensity: ...), tint-block(...)	native	Applies tinting to selected residues.
tintdefault	tint-default(effect)	control	Controls default tint behavior.
emphregion, emphblock	emphasize(sequence, selection, style: ...), emphasis-block(...)	native	Applies emphasis to selected residues.
emphdefault	emphasis-default(style)	control	Controls default emphasis behavior.
lowerregion, lowerblock	lower(sequence, selection), lower-block(...)	control	Renders selected residues in lowercase.

frameblock	frame(sequence, selection, color: ...)	control	Frames selected regions.
feature	mark(position, sequence, selection, ...), motif(...), graph(...)	native	Most hand-written features are easier through <code>mark</code> , <code>motif</code> , and <code>graph</code> ; raw feature styles remain available via lower-level controls.
Feature slots such as <code>top</code> , <code>ttop</code> , <code>bottom</code> , <code>bbottom</code>	mark(position, ...), motif(position: ...), feature-slot-space(position, value)	native	Slot names are preserved as position strings where meaningful.
<code>ttttopspace</code> , <code>tttopspace</code> , <code>ttospace</code> , <code>topspace</code> , <code>bottomspace</code> , <code>bbottomspace</code> , <code>bbbbottomspace</code>	feature-slot-space(position, value)	native	The separate TeX spacing macros are replaced by one explicit slot-space control keyed by position.
featurerule	feature-rule(thickness)	control	Controls boxed feature and structure row stroke thickness.
showfeaturename, showfeaturestylename	feature-text-label(position, name), feature-style-label(position, name)	control	Names descriptive-text rows and feature-style rows.
hidefeaturename, hidefeaturestylename, hidefeaturenames, hidefeaturestylenames	hide-feature-text-label, hide-feature-style-label, hide-feature-text-labels, hide-feature-style-labels	control	Controls feature label visibility.
featurenamecolor, featurestylenamecolor, featurenamescolor, featurestylenamescolor	feature-text-label-color-at, feature-style-label-color-at, feature-text-label-color, feature-style-label-color	control	Controls feature label colors globally or per slot.
Bar graph feature styles	graph(position, sequence, selection, metric, kind: "bar", ...)	native	Built-in metrics include conservation, hydrophobicity, molecular weight, and charge.
Color scale feature styles	graph(..., kind: "color", options: (...))	native	Color-scale tracks support named color ramps and explicit ranges.
Stacked graph styles	graph(..., kind: "stacked", ...)	native	Stacked rows are available for multi-series feature data.
bargraphstretch, colorscalestretch	bar-graph-stretch(value), color-scale-stretch(value)	control	Controls graph track height.
codon	codon(residue, triplets)	control	Custom codon mapping.
geneticcode	genetic-code(name)	control	Selects genetic-code behavior for translation features.
backtranslabel, backtranstext	backtranslation-label(...), backtranslation-text(...)	approx	The biological annotation is preserved. Exact TeX box geometry for horizontal/vertical/oblique/zigzag labels is approximated in Typst layout.

14.8 Sequence Visibility, Ordering, Separators, Layout

TeXshade	Typshade	Status	Migration Notes
hideseq	hide-sequence(sequence)	control	Hides one sequence row.
hideseqs	hide-all-sequences()	control	Hides all sequence rows until selectively reintroduced by other settings.
showseqs	show-all-sequences()	control	Restores sequence row visibility.
killseq	remove-sequence(sequence)	control	Removes a sequence from the displayed alignment model.
donotshade	no-shade(sequences)	control	Displays selected sequences without shading.
orderseqs	sequence-order(order)	control	Reorders sequence rows.
separationline	separation-line(sequence)	control	Adds separator after a sequence.
smallsep, medsep, bigsep, smallsepline, medsepline, bigsepline, nosepline	small-separator(), medium-separator(), large-separator(), no-line-gap()	control	Controls separator thickness/spacing. The older <code>*sepline</code> aliases are covered by the same explicit spacing helpers.
numberingwidth	numbering-width(digits)	control	Reserves numbering column width.
charstretch	character-stretch(value)	control	Controls cell width/stretch.
linestretch	line-stretch(value)	control	Controls row height/stretch.

<code>alignment</code>	<code>alignment-position(position)</code>	<code>control</code>	Controls alignment placement in its containing block. Typshade defaults to <code>left</code> ; accepted values are <code>"left"</code> , <code>"center"</code> , and <code>"right"</code> .
<code>smallblockskip</code> , <code>medblockskip</code> , <code>bigblockskip</code> , <code>noblockskip</code>	<code>small-block-gap()</code> , <code>medium-block-gap()</code> , <code>large-block-gap()</code> , <code>no-block-gap()</code>	<code>control</code>	Block gap sizes are deterministic Typst spacing commands.
<code>vblockspace</code>	<code>block-gap(value)</code>	<code>control</code>	Explicit block gap.
<code>flexblockspace</code> , <code>fixblockspace</code>	<code>flexible-block-gap()</code> , <code>fixed-block-gap()</code>	<code>approx</code>	Commands are represented, but TeX page-breaking glue semantics are not reproduced. Typst uses deterministic layout.
<code>vspace</code> and line-gap-like spacing	<code>line-gap(value)</code> , <code>small-line-gap()</code> , <code>medium-line-gap()</code> , <code>large-line-gap()</code> , <code>no-line-gap()</code>	<code>control</code>	Explicit row/line spacing controls.
<code>alignrightlabels</code> , <code>unalignrightlabels</code>	<code>align-right-labels()</code> , <code>align-left-labels()</code>	<code>control</code>	Controls sequence-label column alignment.
<code>showcaption</code> , <code>shortcaption</code>	<code>caption(text, position: ...)</code> , <code>short-caption(text)</code>	<code>native</code>	Caption content is ordinary Typst content and integrates with document layout.

14.9 Structure Tracks And PDB Selections

TeXshade	Typshade	Status	Migration Notes
<code>includeDSSP</code>	<code>dssp-track(sequence, source)</code>	<code>native</code>	Adds DSSP secondary-structure annotation.
<code>includeSTRIDE</code>	<code>stride-track(sequence, source)</code>	<code>native</code>	Adds STRIDE annotation.
<code>includeHMMTOP</code>	<code>hmmtop-track(sequence, source, source-sequence: ...)</code>	<code>native</code>	Adds HMMTOP topology annotation.
<code>includePHDtopo</code>	<code>phd-topology-track(sequence, source)</code>	<code>native</code>	Adds PHD topology annotation.
<code>includePHDsec</code>	<code>phd-secondary-track(sequence, source)</code>	<code>native</code>	Adds PHD secondary-structure annotation.
TeXshade make new structure-file modes	External preprocessing, then <code>dssp-track</code> / <code>stride-track</code> / <code>hmmtop-track</code>	<code>omitted</code>	Creating new external sources is intentionally outside package rendering. Use a build step, then pass the generated source to Typshade.
<code>showonDSSP</code> , <code>showonSTRIDE</code> , <code>showonHMMTOP</code> , <code>showonPHDtopo</code> , <code>showonPHDsec</code>	<code>show-structure-types(format, types)</code>	<code>control</code>	Controls which structure types are visible for a format.
<code>hideonDSSP</code> , <code>hideonSTRIDE</code> , <code>hideonHMMTOP</code> , <code>hideonPHDtopo</code> , <code>hideonPHDsec</code>	<code>hide-structure-types(format, types)</code>	<code>control</code>	Hides selected structure types.
<code>appearance</code>	<code>structure-appearance(format, structure-type, position, style, text)</code>	<code>control</code>	Controls style and label text for structure annotations.
<code>numcount</code> , <code>alphacount</code> , <code>Alphacount</code> , <code>romancount</code> , <code>Romancount</code>	Template placeholders inside <code>structure-appearance</code> style/text strings.	<code>native</code>	Structure appearance templates expand numeric, alphabetic, and roman counters when rendering repeated structure types.
<code>firstcolumnDSSP</code> , <code>secondcolumnDSSP</code>	<code>use-first-dssp-column()</code> , <code>use-second-dssp-column()</code>	<code>control</code>	Selects which DSSP structure column is used.
<code>printPDBlist</code> , <code>messagePDBlist</code>	<code>pdb-selection(selection)</code>	<code>data</code>	Returns selection-derived residue data/content instead of writing console messages.
PDB point, line, plane selections	<code>pdb-point(...)</code> , <code>pdb-line(...)</code> , <code>pdb-plane(...)</code>	<code>native</code>	Helper functions build readable selection strings for distance-based PDB selections.

14.10 Typography, Themes, And Presets

TeXshade	Typshade	Status	Migration Notes
<code>setfamily</code>	<code>text-family(target, family)</code> or <code>typography(family: ...)</code>	<code>native</code>	Targets include alignment parts such as names, numbers, residues, consensus, ruler, and features.

setseries	text-weight(target, weight) or typography(weight: ...)	native	Uses Typst's text weight model instead of TeX font series names.
setshape	text-posture(target, posture) or typography(posture: ...)	native	Uses Typst posture/style semantics.
setsize	text-size(target, size) or typography(size: ...)	native	Can set absolute Typst lengths or package size keywords where supported.
setfont	text-style(target, family, weight, posture, size)	native	One command can set a complete text style for a target.
featuresrm, featuressf, featuresstt, featuresbf, featuresmd, featuresit, featuressl, featuressc, featuresup, featurestiny, featuresscriptsize, featuresfootnotesize, featuressmall, featuresnormalsize, featureslarge, featuresLarge, featuresLARGE, featureshuge, featuresHuge	typography(target: "features", ...)	native	TeX's many short font macros are intentionally collapsed into target-based Typst typography controls.
namesrm, numberingrm, residuesrm, rulerrm, legendrm, featurenamesrm, featurestylenames, featurestylenamesrm, and corresponding family/shape/size variants	typography(target: ..., ...), text-family, text-weight, text-posture, text-size	native	All target-specific TeX font shortcuts map to the same target-based text API.
Global document visual style	theme: "screen", theme: shade-theme(...), theme: visual-theme(...)	native	Themes are explicit values, not ambient macro state.
TeXshade preset-like command blocks	preset: "publication" or shade-preset(name)	native	Reusable presets are ordinary Typst values and can be imported from local files.

The legacy TeXshade typography shortcuts follow a regular naming scheme. Typshade maps each family to `typography(target: ..., ...)`, `text-family`, `text-weight`, `text-posture`, or `text-size`.

TeXshade shortcut family	Covered suffixes
features*	rm, sf, tt, bf, md, it, sl, sc, up, tiny, scriptsize, footnotesize, small, normalsize, large, Large, LARGE, huge, Huge
featurestyles*	rm, sf, tt, bf, md, it, sl, sc, up, tiny, scriptsize, footnotesize, small, normalsize, large, Large, LARGE, huge, Huge
featurenames*, featurestylenames*	sf, tt, bf, md, it, sl, sc, up, tiny, scriptsize, footnotesize, small, normalsize, large, Large, LARGE, huge, Huge
names*, numbering*, residues*, legend*	sf, tt, bf, md, it, sl, sc, up, tiny, scriptsize, footnotesize, small, normalsize, large, Large, LARGE, huge, Huge
ruler*, rulename*	sf, tt, tiny, scriptsize, footnotesize, small, normalsize, large, Large, LARGE, huge, Huge; rulename* also includes rm

14.11 Utility And Analysis Functions

TeXshade	Typshade	Status	Migration Notes
molweight	molecular-weight(sequence, unit: "Da")	native	Standalone sequence helper. Graph tracks can also use molecular-weight-derived metrics.
charge	net-charge(sequence, termini: "o")	native	Standalone sequence helper. Graph tracks can also use charge metrics.

percentsimilarity	percent-similarity(source, sequence-a, sequence-b, ...)	native	Computes pairwise percent similarity for an alignment source and optional displayed selection.
percentidentity	percent-identity(source, sequence-a, sequence-b, ...)	native	Computes pairwise percent identity over shared non-gap positions.
similaritytable, identitytable	similarity-table(source, ...)	native	Returns a Typst table with similarity values above the diagonal and identity values below the diagonal, matching TeXshade’s combined table concept.
Current-alignment macro access	alignment-data(source), parse-alignment(text)	data	The parsed alignment is explicit data that can be reused in Typst logic.
Sequence list diagnostics	sequence-list(source)	data	Document-facing sequence index/name table.
Selection diagnostics	selection-preview(source, sequence, selection), selection-table(source, ...)	data	Preview motif/range/PDB selections without relying on TeX log output.

14.12 External Scripts And Side Effects

TeXshade includes helper behavior that prints messages, writes files, or generates external helper script text. Typshade deliberately avoids hidden render-time side effects:

TeXshade	Typshade	Status	Migration Notes
exportconsensus	Use alignment-data or an explicit build script.	omitted	No arbitrary file writes during Typst package evaluation.
messagePDBlist	pdb-selection(...) as content/data	data	The information is available, but it is not sent to TeX’s console.
printPDBlist	pdb-selection(...) or selection-table(...)	data	Use document content rather than TeX output streams.
structurememe, memeStandardcolors, memeRed, memeYellow, memeBlue, memeWhite, memeBlack	External script generation outside the package.	omitted	Structure meme rendering requires generating Chimera-style command files; this side-effect belongs in an external build/tooling layer.
chimerachain, chimeraBallScale, memelabelcutoff, chimeraaxisdistance, chimeraaxisdistance, echostructurefile	External script generation outside the package.	omitted	These configure TeXshade’s Chimera command-file output and are intentionally not public Typshade document-rendering controls.
Driver-dependent specials	Typst-native rendering only.	omitted	DVI/PDF driver hooks and TeX specials are not part of the Typst package model.

14.13 Complete Feature Coverage Summary

Typshade provides the TeXshade feature set through these current public groups:

Feature Family	Current Typshade Coverage
Alignment rendering	shade, format:, seq-type:, commands:, figure:, regions:, features:.
Purpose-level figures	publication, motif-map, structure-map, logo-analysis, overview.
Scoring and shading	identical, similar, diverse, functional, single-sequence, tcoffee, scoring-mode, threshold, color-scheme, residue-style, matrix/weight controls.
Sequence display	lines, window, names, numbers, label and numbering controls, sequence visibility/order controls, gap controls, residue hiding, fingerprint controls.
Consensus and ruler tracks	consensus, consensus-track, ruler, ruler-track, symbols/colors/language/marker/rotation controls.
Logos and legends	logo, sequence-logo, subfamily-logo, frequency correction, negative values, relevance markers, logo colors, logo scale, legend controls.
Regions and annotations	highlight, tint, emphasize, lower, frame, motif, mark, graph, feature label and graph controls.
Translation features	codon, genetic-code, backtranslation-label, backtranslation-text, translation/complement feature styles.
Structure tracks	structures, structure-tracks, dssp-track, stride-track, hmmtrack, phd-topology-track, phd-secondary-track, structure visibility and appearance controls.

PDB selections	pdb-point, pdb-line, pdb-plane, pdb-selection, selection-preview, selection-table.
Typography and layout	typography, text-family, text-weight, text-posture, text-size, text-style, spacing, separator, block-gap, line-gap, caption controls.
Analysis utilities	alignment-data, parse-alignment, alignment-summary, sequence-list, selection-preview, selection-table, percent-identity, percent-similarity, similarity-table, molecular-weight, net-charge.

15 Public API Index

This index is organized from the Typshade side. Use it when you know the kind of object you want to control but do not remember the exact helper name.

Group	Public Names
Renderer	shade
Recipes	publication, motif-map, structure-map, logo-analysis, overview
Presets and themes	shade-preset, shade-theme, visual-theme, resolve-color, scale-color
Inspection and data	alignment-data, parse-alignment, alignment-summary, selection-preview, sequence-list, selection-table, percent-identity, percent-similarity, similarity-table
Scoring shortcuts	identical, similar, diverse, functional, single-sequence, tcoffee
Layout shortcuts	lines, window, names, no-names, numbers, no-numbers, no-numbering, typography, gap-style
Track shortcuts	consensus, no-consensus, ruler, no-ruler, logo, no-logo, legend, no-legend, structures
Track controls	consensus-track, ruler-track, ruler-marker, sequence-logo, no-sequence-logo, subfamily-logo, no-subfamily-logo, legend-track, structure-tracks
Annotations	highlight, tint, emphasize, mark, motif, graph, pdb-point, pdb-line, pdb-plane
Core controls	sequence-type, color-scheme, scoring-mode, tcoffee-scores, sequence-window, residues-per-line, threshold
Conservation controls	shade-all-residues, all-match-threshold, disable-all-match-threshold, hide-all-match-positions, show-all-match-positions, weight-table, set-weight, gap-penalty
Residue/group controls	residue-style, peptide-groups, dna-groups, peptide-similarities, dna-similarities, clear-functional-groups, functional-group, functional-style
Names and numbering	names-track, numbering-track, sequence-name, names-color, sequence-name-color, hide-sequence-name, numbering-color, sequence-number-color, hide-sequence-number, start-number, allow-zero-numbering, disallow-zero-numbering, sequence-length
Consensus/ruler details	consensus-name, consensus-language, consensus-symbols, consensus-colors, consensus-from-sequence, consensus-from-all-sequences, ruler-steps, ruler-color, ruler-name, ruler-name-color, ruler-space, rotate-ruler, unrotate-ruler
Gap and sequence display	gap-char, gap-rule, gap-colors, stop-char, show-leading-gaps, hide-leading-gaps, hide-residues, show-residues, keep-single-sequence-gaps, shift-single-sequence
Regions and sequence rows	domain, domain-gap-rule, domain-gap-colors, highlight-block, region-color-scheme, lower, lower-block, emphasis-block, tint-block, tint-default, emphasis-default, frame, hide-sequence, hide-all-sequences, show-all-sequences, remove-sequence, no-shade, separation-line, sequence-order
Feature and translation controls	feature-rule, codon, genetic-code, backtranslation-label, backtranslation-text, feature-text-label, feature-style-label, hide-feature-text-label, hide-feature-style-label, hide-feature-text-labels, hide-feature-style-labels, feature-text-label-color, feature-style-label-color, feature-text-label-color-at, feature-style-label-color-at
Logo controls	frequency-correction, no-frequency-correction, subfamily, sequence-logo-name, subfamily-logo-name, logo-scale, no-logo-scale, logo-stretch, negative-logo-values, no-negative-logo-values, relevance-threshold, relevance-marker, no-relevance-marker, logo-color, clear-logo-colors
Legend controls	legend-color, legend-offset, color-swatch
Structure controls	show-structure-types, hide-structure-types, structure-appearance, use-first-dssp-column, use-second-dssp-column, stride-track, dssp-track, hmtop-track, phd-topology-track, phd-secondary-track
Graph/layout controls	bar-graph-stretch, color-scale-stretch, alignment-position, character-stretch, line-stretch, numbering-width, fingerprint, align-right-labels, align-left-labels. Default block placement is left-aligned.

Typography and spacing	text-family, text-weight, text-posture, text-size, text-style, caption, short-caption, small-separator, medium-separator, large-separator, no-block-gap, small-block-gap, medium-block-gap, large-block-gap, block-gap, flexible-block-gap, fixed-block-gap, no-line-gap, small-line-gap, medium-line-gap, large-line-gap, line-gap, feature-slot-space
Sequence utilities	molecular-weight, net-charge, pdb-selection

16 Public API Example Catalog

This catalog gives an example invocation for every public Typshade function. The examples are intentionally compact; the visual behavior of the larger families is shown in the worked examples above. When a subsection lists many invocations, the result caption names the compact call it demonstrates; the remaining snippets are command-surface examples.

16.1 Rendering, Recipes, Presets, Themes, And Colors

Use `shade` for rendering, recipe functions for common figure purposes, presets and themes for reusable visual defaults, and color helpers when building custom Typst logic around Typshade colors.

```
#let alignment = read("alignment.msff", encoding: none)

#shade(alignment, format: "msff")

#shade(alignment, format: "msff", figure: publication(region: "80..112"))
#shade(alignment, format: "msff", figure: motif-map(("NPA": "loop")))
#shade(alignment, format: "msff", figure: structure-map(1, topology: read("AQP1.top", encoding: none)))
#shade(alignment, format: "msff", figure: logo-analysis(sequence: 3, region: "203..235"))
#shade(alignment, format: "msff", figure: overview(colors: "grays"))
#let resolved-figure = resolve-figure(
  alignment,
  "msff",
  publication(region: "80..112"),
)

#shade(alignment, format: "msff", preset: "publication")
#shade(alignment, format: "msff", theme: "screen")
#shade(
  alignment,
  format: "msff",
  theme: visual-theme(names: "RoyalBlue", ruler: "DarkGray"),
)

#let preset-commands = shade-preset("publication")
#let theme-commands = shade-theme((names: "RoyalBlue", ruler: "DarkGray"))
#let resolved-red = resolve-color("BrickRed")
#let midpoint = scale-color("ColdHot", 50)
```

Typeset result publication(...) recipe with mode: "similar" and region: "80..112"

	80	90	100	110	
AQP1.PRO	TLGLL	SCQ	ISILRAVMYII	AQC	VGAIVASAIL 112
AQP2.PRO	TVAC	LVGCHV	SFLRAAFYVA	AQL	LGAVAGAAIL 104
AQP3.PRO	TFAM	CFLAREPW	IKLPITYTL	AQT	LGAF LGAGIV 112
AQP4.PRO	TVAMV	CTRKI	SIIAKSVFYIT	AQC	LGAIIGAGIL 133
AQP5.PRO	TLALL	IGNQ	ISLLRAVFYVA	AQL	VGAIA GAGIL 105
consensus	! * *	** * * * *	!! * ! ! *	* * * ! *	

16.2 Data, Inspection, And Analysis

These functions return parsed alignment data, diagnostic tables, or numerical sequence comparisons. They are useful both in prose and in figure-building logic.

```
#let alignment = read("alignment.msff", encoding: none)
#let parsed = alignment-data(alignment, format: "msff")
```

```

#let tiny = parse-alignment(">A\nACD\n>B\nA-D", format: "fasta")

#alignment-summary(alignment, format: "msf")
#sequence-list(alignment, format: "msf")
#selection-preview(alignment, 1, "80..112", format: "msf")
#selection-table(
  alignment,
  "80..112",
  "NPA",
  format: "msf",
  sequence: 1,
)

#percent-identity(alignment, 1, 2, format: "msf")
#percent-similarity(alignment, 1, 2, format: "msf")
#similarity-table(alignment, format: "msf")

```

Typeset result alignment-summary(...) followed by selection-preview(...)

Format	MSF
Type	P
Sequences	5
Columns	354
Names	AQP1.PRO, AQP2.PRO, AQP3.PRO, AQP4.PRO, AQP5.PRO

80,81,82,83,84,85,86,87,88,89,90

16.3 Scoring, Shading, And Sequence-Type Controls

Scoring helpers are usually placed in `commands:`. The lower-level controls are available when a figure needs exact threshold, residue style, similarity, or weight-table behavior.

```

#let alignment = read("alignment.msfa", encoding: none)
#let tcoffee-data = read("AQP_TC.asc", encoding: none)

```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    sequence-type("P"),
    identical(colors: "blues", threshold: 50),
    similar(colors: "greens", threshold: 45),
    diverse(sequence: 1),
    functional("charge"),
    single-sequence(sequence: 1),
    tcoffee(tcoffee-data),
    scoring-mode("similar"),
    tcoffee-scores(tcoffee-data),
    color-scheme("reds"),
    threshold(60),
    all-match-threshold(value: 90),
    disable-all-match-threshold(),
    shade-all-residues(),
    hide-all-match-positions(),
    show-all-match-positions(),
    weight-table("BLOSUM62"),
    set-weight("E", "Q", 2),
    gap-penalty(-5),
  ),
)

```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    residue-style("conserved", "White", "RoyalBlue", case: "upper", style: "bold"),
    peptide-groups(("FYW", "ILVM", "RK", "DE", "GA", "ST", "NQ")),
  ),
)

```

```

dna-groups(("GAR", "CTY")),
peptide-similarities("S", "TA"),
dna-similarities("A", "GR"),
clear-functional-groups(),
functional-group("acidic (-)", "DE", "White", "Red"),
functional-style("D", "White", "BrickRed"),
),
)

```

Typeset result functional("charge"), window(...), and legend()

 acidic (-)
 basic (+)

```

AQP1.PRO  GLGI E IIGTLQLVLCVLATT DRRRRD LGGSAPL 170
AQP2.PRO  AVTV E LFLTMQLVLCIFAST DERRG DNLGSPAL 162
AQP3.PRO  GFF D QFIGTAALIVCVLAIV DPNNPVP RGL EAF 186
AQP4.PRO  GLLV E LIITFQLVFTIFASC DSKRT DVTGSVAL 191
AQP5.PRO  AMVV E ILTFQLALCIFSS DSRRTSPVGSPAL 163
consensus *   **** ! *!*****! ** *   !* **

```

16.4 Windows, Labels, Numbering, Rulers, And Gaps

The shortcut functions cover common cases. The track/control functions expose individual knobs for sequence labels, numbering, ruler names, ruler markers, gap characters, stop symbols, and leading-gap behavior.

```

#shade(
  alignment,
  format: "msf",
  commands: (
    lines(40),
    residues-per-line(40),
    window(1, "80..112"),
    sequence-window(1, "80..112", start: 80),

    names(position: "left", color: "RoyalBlue"),
    no-names(),
    numbers(position: "right", color: "DarkGray"),
    no-numbers(),
    names-track(position: "left", color: "RoyalBlue"),
    no-numbering(),
    numbering-track(position: "leftright", color: "DarkGray"),

    sequence-name(1, "AQP1"),
    names-color("RoyalBlue"),
    sequence-name-color((1, 3), "Red"),
    hide-sequence-name((2,)),
    numbering-color("DarkGray"),
    sequence-number-color((1, 3), "Red"),
    hide-sequence-number((2,)),

    start-number(1, 80, selection: "80..112"),
    allow-zero-numbering(),
    disallow-zero-numbering(),
    sequence-length(1, 269),
  ),
)

```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    ruler("top", sequence: 1, every: 10, name: "AQP1 positions"),
    no-ruler("bottom"),
    ruler-track(position: "bottom", sequence: 1, steps: 5, color: "DarkGray"),
    ruler-marker(100, "site", position: "top", color: "Red"),

```

```

ruler-steps(10),
ruler-color("DarkGray"),
ruler-name("positions"),
ruler-name-color("Red"),
ruler-space(2pt),
rotate-ruler(),
unrotate-ruler(),

gap-style(foreground: "Gray60", background: "White", rule: 0.5pt),
gap-char("."),
gap-rule(0.5pt),
gap-colors("Gray60", "White"),
stop-char("*"),
show-leading-gaps(),
hide-leading-gaps(),
),
)

```

Typeset result window(...), ruler(...), names(...), and numbers(...)

	80	90	100	110		
AQP1.PRO	TLGL	LLSCQ	ISILRAVMYII	AQCVGAT	VASAIL 112	
AQP2.PRO	TVACL	VGCHV	SFLRAAFYVA	AQLLGA	VAGAAIL 104	
AQP3.PRO	TFAMC	FLAREP	WIKLPITYTL	AQTLGA	FLGAGIV 112	
AQP4.PRO	TVAMV	CTRK	ISIAKS	VFYIT	AQCLGAT	IAGAIL 133
AQP5.PRO	TLALL	IGNQ	ISLLRAVFY	VAAQLV	GATAGAIL 105	

16.5 Consensus, Logos, Legends, And Swatches

Consensus, logo, and legend helpers can be used as shortcuts or as explicit tracks. Logo controls cover frequency correction, subfamily views, scale labels, negative values, relevance markers, and custom residue colors.

```

#shade(
  alignment,
  format: "msf",
  commands: (
    consensus("bottom", scale: "ColdHot", name: "conservation"),
    no-consensus(),
    consensus-track(position: "top", scale: "BlueRed", name: "consensus"),
    consensus-name("agreement"),
    consensus-language("english"),
    consensus-symbols(".", "lower", "upper"),
    consensus-colors(
      none-fg: "Black",
      none-bg: "White",
      conserved-fg: "White",
      conserved-bg: "RoyalBlue",
      allmatch-fg: "Yellow",
      allmatch-bg: "RoyalPurple",
    ),
    consensus-from-sequence(1),
    consensus-from-all-sequences(),
  ),
)

```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    logo("top", colors: "charge", name: "logo"),
    no-logo(),
    sequence-logo(position: "top", colors: "rasmol", name: "logo"),
    no-sequence-logo(),
    subfamily-logo((3,), position: "bottom", colors: "charge", name: "AQP3", negative-name:
"others"),
  ),
)

```



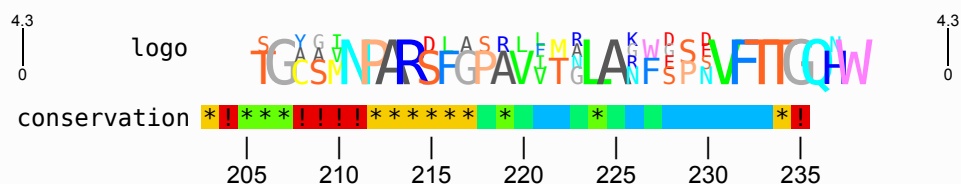
```

no-subfamily-logo(),
frequency-correction(),
no-frequency-correction(),
subfamily((3,)),
sequence-logo-name("sequence logo"),
subfamily-logo-name("AQP3", negative-name: "others"),
logo-scale(position: "leftright", color: "DarkGray"),
no-logo-scale(),
logo-stretch(1.2),
negative-logo-values(),
no-negative-logo-values(),
relevance-threshold(1.0),
relevance-marker(char: "*", color: "Red"),
no-relevance-marker(),
logo-color("DE", "Red"),
clear-logo-colors(default: "Black"),
),
)

#shade(alignment, format: "msf", commands: (legend(color: "Black"),))
#shade(alignment, format: "msf", commands: (no-legend(), legend-track(color: "Black")))
#shade(alignment, format: "msf", commands: (legend-color("DarkGray"), legend-offset(6pt, 0pt)))
#color-swatch("RoyalBlue")

```

Typeset result logo(...), consensus(...), hidden sequences, and bottom ruler



16.6 Regions, Domains, Motifs, Marks, Graphs, And PDB Selections

Region and annotation helpers handle the common TeXshade feature patterns: selected domains, block highlighting, tinting, emphasis, lowercase regions, frames, marks, motifs, graph tracks, and structure-derived selections.

```

#let pdb = read("1J4N.pdb", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    domain(1, "80..90,100..112"),
    domain-gap-rule(0.6pt),
    domain-gap-colors("Gray70", "White"),

    highlight(1, "NPA", fg: "White", bg: "BrickRed"),
    highlight-block(1, "80..112", "White", "LightBlue"),
    region-color-scheme(1, "80..112", "reds"),
    tint(1, "158..163", intensity: "strong"),
    tint-block(1, "158..163", intensity: "weak"),
    tint-default("normal"),
    emphasize(1, "NPA", style: "italic"),
    emphasis-block(1, "158..163", style: "bold"),
    emphasis-default("italic"),
    lower(1, "80..90"),
    lower-block(1, "80..90"),
    frame(1, "82..82,106..106", color: "Red"),
  ),
)

```

```

#shade(
  alignment,
  format: "msf",

```

```

commands: (
  mark("top", 1, "93..93", fill: "Yellow", text: "site"),
  motif(1, "NX[ST]N", text: "glycosylation", position: "top"),
  graph("top", 1, "138..170", "conservation", kind: "bar"),
  graph("bottom", 1, "138..170", "hydrophobicity", kind: "color", options: ("ColdHot",)),
  bar-graph-stretch(1.4),
  color-scale-stretch(1.2),
  feature-rule(0.7pt),
  feature-text-label("top", "features"),
  feature-style-label("bottom", "graph"),
  hide-feature-text-label("top"),
  hide-feature-style-label("bottom"),
  hide-feature-text-labels(),
  hide-feature-style-labels(),
  feature-text-label-color("DarkGray"),
  feature-style-label-color("DarkGray"),
  feature-text-label-color-at("top", "Red"),
  feature-style-label-color-at("bottom", "Blue"),
),
)

```

```

#let point = pdb-point(pdb, 81, distance: 8, atom: "CA")
#let line = pdb-line(pdb, 81, 168, distance: 1)
#let plane = pdb-plane(pdb, 66, 73, 199, distance: 1)

#shade(
  alignment,
  format: "msf",
  commands: (
    window(1, point),
    highlight(1, line, bg: "Yellow"),
    mark("top", 1, plane, text: "plane"),
  ),
)

#pdb-selection(point)

```

Typeset result `pdb-point(...)` selection with `highlight(...)` and `motif(...)`

```

      1 2 3
      | | |
Alpha A E F . 3
Beta  A D F . 3
Gamma A . G . 2
seq4  A C . . 2

```

16.7 Sequence Visibility, Ordering, Layout, Typography, And Spacing

These controls cover sequence visibility, row ordering, separators, figure placement, character metrics, fingerprints, label alignment, target-specific typography, captions, and block/line/feature spacing.

```

#shade(
  alignment,
  format: "msf",
  commands: (
    hide-sequence(2),
    hide-all-sequences(),
    show-all-sequences(),
    remove-sequence(5),
    no-shade((1, 3)),
    separation-line(3),
    sequence-order((5, 4, 3, 2, 1)),

    hide-residues(),
    show-residues(),
    fingerprint(360),
    alignment-position("left"),
    character-stretch(1.1),
  ),
)

```

```

    line-stretch(1.05),
    numbering-width(6),
    align-right-labels(),
    align-left-labels(),
  ),
)

```

```

#shade(
  alignment,
  format: "msf",
  commands: (
    typography(target: "all", family: "mono", size: 8pt),
    text-family("names", "mono"),
    text-weight("names", "bold"),
    text-posture("features", "italic"),
    text-size("residues", 7pt),
    text-style("ruler", "sans", "regular", "normal", 7pt),

    caption([Aquaporin alignment], position: "bottom"),
    short-caption([AQP alignment]),
    small-separator(),
    medium-separator(),
    large-separator(),
    no-block-gap(),
    small-block-gap(),
    medium-block-gap(),
    large-block-gap(),
    block-gap(0.8em),
    flexible-block-gap(),
    fixed-block-gap(),
    no-line-gap(),
    small-line-gap(),
    medium-line-gap(),
    large-line-gap(),
    line-gap(0.2pt),
    feature-slot-space("top", 2pt),
  ),
)

```

Typeset result typography(...), character metrics, left placement, and ruler

	80	90	100	110
AQP1.PRO	T L G L L L S C Q I S I L R A V M Y I I A Q C V G A I V A S A I L	112		
AQP2.PRO	T V A C L V G C H V S F L R A A F Y V A A Q L L G A V A G A A I L	104		
AQP3.PRO	T F A M C F L A R E P W I K L P I Y T L A Q T L G A F L G A G I V	112		
AQP4.PRO	T V A M V C T R K I S I A K S V F Y I T A Q C L G A I I G A G I L	133		
AQP5.PRO	T L A L L I G N Q I S L L R A V F Y V A A Q L V G A I A G A G I L	105		

16.8 Translation, Structure Tracks, And Sequence Utilities

Translation helpers configure codon handling and backtranslation display. Structure helpers attach DSSP, STRIDE, HMMTOP, PHD topology, or PHD secondary structure data and control which structure classes are visible.

```

#let dna = read("dna-alignment.msf", encoding: none)

#shade(
  dna,
  format: "msf",
  seq-type: "N",
  commands: (
    single-sequence(sequence: 1),

```

```

keep-single-sequence-gaps(),
shift-single-sequence(value: -1),
codon("A", "GCA,GCG,GCC,GCT,GCU,GCN"),
genetic-code("standard"),
backtranslation-label(style: "oblique", size: "tiny"),
backtranslation-text(style: "horizontal", size: "tiny"),
mark("bottom", 1, "1..90", style: "translate[Red]", text: "translation"),
mark("bottom", 1, "1..90", style: "complement[LightBlue][lower]", text: "complement"),
),
)

```

```

#let topology = read("AQP1.top", encoding: none)
#let secondary = read("AQP1.phd", encoding: none)
#let hmmtop = read("AQP_HMM.ext", encoding: none)
#let dssp = read("model.dssp", encoding: none)
#let stride = read("model.stride", encoding: none)

#shade(
  alignment,
  format: "msf",
  commands: (
    structures(
      1,
      topology: topology,
      secondary: secondary,
      hmmtop: hmmtop,
    ),
    structure-tracks(
      1,
      topology: topology,
      secondary: secondary,
      hmmtop: hmmtop,
    ),
    phd-topology-track(1, topology),
    phd-secondary-track(1, secondary),
    hmmtop-track(1, hmmtop, source-sequence: 1),
    dssp-track(1, dssp),
    stride-track(1, stride),
    show-structure-types(
      "PHDtopo",
      ("TM", "internal", "external"),
    ),
    hide-structure-types("PHDtopo", ("internal",)),
    structure-appearance(
      "PHDtopo",
      "TM",
      "top",
      "box[Blue]:TM",
      "",
    ),
    use-first-dssp-column(),
    use-second-dssp-column(),
  ),
)

```

```

#molecular-weight("ACDEFGHIK", unit: "Da")
#molecular-weight("ACDEFGHIK", unit: "kDa")
#net-charge("ACDEFGHIK", termini: "o")
#net-charge("ACDEFGHIK", termini: "i")

```

Typeset result single-sequence(...) with translation and complement feature marks

AQP1nuc.SEQ CCTGGGCATTGAGATCATTGGCACCCCTGCA 443

17 Typst-Specific Improvements

- **figure:** recipes let hand-written figures read as scientific intent.
- **commands:** keeps custom figures explicit without adding another top-level API.

- Dictionaries and arrays make presets easy to share.
- Inspection helpers such as `alignment-summary`, `sequence-list`, and `selection-table` can be embedded in documents.
- `alignment-data` and `parse-alignment` expose parsed data for custom Typst logic.
- Public names follow Typst conventions instead of TeX macro naming.

18 Known Typst Differences

Some TeXshade behavior depends on TeX box construction, console messages, or external file writes. Typshade keeps equivalent document-facing behavior where it is meaningful, but does not promise byte-for-byte or box-for-box output.

Area	Typshade behavior
External file writes	Package evaluation does not perform arbitrary side-effect exports. Return data/content or use external tooling when needed.
Console messages	Inspection helpers return Typst content/data instead of writing TeX messages.
Page breaking glue	Typst uses deterministic layout controls rather than TeX glue side effects.
Exact box geometry	Ruler and backtranslation geometry is represented in Typst layout, not TeX box primitives.

19 License

This project is distributed under the GPL v2 License.